JavaScript Array Method - slice()

The `slice()` method is one of the most useful array methods in JavaScript. It is used to **copy a portion of an array** into a new array without modifying the original array.

Unlike `splice()`, the `slice()` method **does not change the original array**.

Table of Contents

What is slice()?

Definition

The `slice()` method returns a **shallow copy** of a selected portion of an array.

It copies elements from the original array into a **new array**.

The original array remains unchanged.

Why Use slice()?

We use `slice()` when we need to:

- Copy part of an array
- Get specific elements
- Create a backup of an array
- Display paginated data
- Avoid modifying the original array

Syntax

```
array.slice(start);

array.slice(start, end);
```

Parameters

Parameter	Description
start	Starting index (included)
end	Ending index (excluded, optional)

Return Value

Returns a **new array** containing the selected elements.

The original array is **not modified**.

How slice() Works

Array

```
["Apple","Mango","Orange","Banana"]
```

↓

```
slice(1,3)
```

↓

New Array

```
["Mango","Orange"]
```

Original Array

```
["Apple","Mango","Orange","Banana"]
```

Example 1 (Basic)

```
const fruits = [
    "Apple",
    "Mango",
    "Orange",
    "Banana"
];
const result = fruits.slice(1,3);
console.log(result);
console.log(fruits);
```

Output

```
["Mango","Orange"]

["Apple","Mango","Orange","Banana"]
```

Explanation

- Starts from index **1**
- Stops before index **3**
- Returns a new array
- Original array remains unchanged

Example 2 (Only Start Index)

```
const numbers = [
    10,
    20,
    30,
    40,
    50
];
console.log(numbers.slice(2));
```

Output

```
[30,40,50]
```

Example 3 (Copy Entire Array)

```
const colors = [
    "Red",
    "Green",
    "Blue"
];
const copy = colors.slice();
console.log(copy);
```

Output

```
["Red","Green","Blue"]
```

Example 4 (Negative Index)

```
const numbers = [
    10,
    20,
    30,
    40,
    50
];
console.log(numbers.slice(-2));
```

Output

```
[40,50]
```

Explanation

Negative indexes count from the end.

```
-1 → Last element

-2 → Second last element
```

Example 5 (Negative Start and End)

```
const numbers = [
    10,
    20,
    30,
    40,
    50
];
console.log(numbers.slice(-4,-1));
```

Output

```
[20,30,40]
```

Example 6 (Empty Result)

```
const numbers = [
    10,
    20,
    30
];
console.log(numbers.slice(5));
```

Output

```
[]
```

Example 7 (Objects)

```
const users = [
    {
        name:"John"
    },
    {
        name:"Sara"
    },
    {
        name:"David"
```

```
    }

];

const result = users.slice(0,2);

console.log(result);
```

Output

```
[
    { name: "John" },
    { name: "Sara" }
]
```

Example 8 (Strings in Array)

```
const cities = [

    "Dhaka",

    "Chattogram",

    "Rajshahi",

    "Khulna"

];

console.log(cities.slice(1,3));
```

Output

```
["Chattogram","Rajshahi"]
```

Real Project Example 1

Pagination

```
const products = [
    "Laptop",
    "Mouse",
    "Keyboard",
    "Monitor",
    "Speaker"
];
const pageOne = products.slice(0,2);
console.log(pageOne);
```

Output

```
["Laptop","Mouse"]
```

Real Project Example 2

Backup Array

```
const students = [
    "Rahim",
    "Karim",
```

```
    "Rokon"
];
const backup = students.slice();
console.log(backup);
```

Output

```
["Rahim","Karim","Rokon"]
```

Real Project Example 3

Display Latest News

```
const news = [
    "News 1",
    "News 2",
    "News 3",
    "News 4"
];
const latest = news.slice(-2);
console.log(latest);
```

Output

```
["News 3","News 4"]
```

Real Project Example 4

Featured Products

```
const products = [
    "Laptop",
    "Mouse",
    "Keyboard",
    "Monitor"
];
const featured = products.slice(0,3);
console.log(featured);
```

Output

```
["Laptop","Mouse","Keyboard"]
```

Real Project Example 5

Top Scores

```
const scores = [
```

```
    98,
    95,
    90,
    85,
    80
];
const topThree = scores.slice(0,3);
console.log(topThree);
```

Output

```
[98,95,90]
```

Best Practices

Use `slice()` when you don't want to modify the original array.

Use `slice()` to create a copy of an array.

```
const copy = array.slice();
```

Use negative indexes to get the last elements.

```
array.slice(-3);
```

Use meaningful variable names.

```
copy

backup

featuredProducts

latestNews
```

Common Mistakes

Mistake 1

Thinking `slice()` modifies the original array.

Wrong

```
const numbers = [1,2,3];

numbers.slice(1);

console.log(numbers);
```

Output

```
[1,2,3]
```

The original array remains unchanged.

Mistake 2

Confusing `slice()` with `splice()` .

```
slice()
```

Copies elements.

```
splice()
```

Removes or inserts elements.

Mistake 3

Forgetting that the ending index is excluded.

```
const arr = [10,20,30,40];

arr.slice(1,3);
```

Output

```
[20,30]
```

Index **3** is **not included**.

Mistake 4

Using an invalid start index.

```
const arr = [1,2,3];

arr.slice(10);
```

Output

```
[]
```

Interview Questions

What does `slice()` do?

It returns a copy of part of an array.

Does `slice()` modify the original array?

No.

What does `slice()` return?

A new array.

Is the ending index included?

No.

The ending index is excluded.

Can `slice()` use negative indexes?

Yes.

Which method modifies the original array?

```
splice()
```

Which method copies an array?

```
slice()
```

Summary Table

Feature	Description
Method	`slice()`
Purpose	Copy part of an array
Parameters	start, end (optional)
Returns	New array
Changes Original Array	No
Supports Negative Index	Yes
Common Uses	Copy, Backup, Pagination, Filtering

Quick Comparison

Method	Returns	Changes Original Array
`slice()`	New Array	No
`splice()`	Removed Elements	Yes

Final Notes

The `slice()` method is one of the safest array methods because it **never changes the original array**. It is widely used in:

- React applications
- Pagination
- Data backup
- Copying arrays

- Filtering displayed data
- API response handling

Mastering `slice()` is essential before learning `splice()`, `map()`, `filter()`, and other advanced array methods.